RishteyZindagi.com

Website Test Report & Developer Handover Guide

Local setup · OTP · Deployment · Project workflow (flow charts)

Date	22 October 2026
Project	Next.js 14 (App Router) + TypeScript + PostgreSQL + Drizzle ORM
Repository	`github.com/uiuxathar123/RishteyZindagi-Website10-08-2026` (private)
Code base	Last commit `b25c858` + uncommitted work (see section 7 — must be committed first)
Tested on	Windows 11, Node v24, PostgreSQL 18 (local), Microsoft Edge

> **Yeh document kis liye hai?** Part A batata hai ki website kitni tez hai aur kya-kya test hua. Part B ek naye developer ko bina kisi madad ke project ko apne computer par chalane aur server par deploy karne ke steps deta hai. Part C mein poore project ke flow charts hain.

1. Part A — Test Report (sections 1–6)
2. Part B — Developer Handover (sections 7–15)
3. Part C — Project Flow Charts (sections 16–20)

Part A — Website Test Report

Practical testing: speed, page-to-page navigation, a real account, photos and WebP

1. Short summary

Area	Result	Notes
Speed for real users (production build)	Very good	Median page ~29 ms, slowest ~89 ms, every page ≤ 126 kB JavaScript
Speed on local dev (`npm run dev`)	Normal for dev	First open of a page 1–8 s (compiling), after that 26–443 ms
Signup → profile → photo → approval → search	Working	Tested with a new account end to end
Member features (search, recommendations, shortlist, interests, chat, notifications)	Working	Interest correctly needs an active membership
Photos in WebP	Done	All 116 member photos WebP; new uploads auto-convert; site images served as WebP
Super Admin: Personal Matchmaking, Members, Introductions, Company Verified, Users panel	Working	Read-only and real-flow checks done
Automated tests (focused sets)	Pass	151 matchmaking/payments/security tests + 39 membership/search/avatar tests
Real vendors (SMS/e-mail OTP, Razorpay live, S3)	Not tested	Need real credentials — only local mocks/dev modes were used

2. Speed — numbers

2.1 Production build (what real users get)

Measure	Result
Page response, median (warm)	**29 ms**
Page response, median (first visit)	32 ms
Slowest public page	89 ms (Success Stories)
APIs (search, offer, plans)	30–70 ms
Biggest page JavaScript (first load)	126 kB (My Profile); public pages ~96–101 kB
Server start	0.5 s

2.2 Page-to-page navigation in the browser (local dev, real clicks)

Pages	First time (dev compiles the page)	Every time after
Dashboard: Profile, Matches, Interests, Shortlist, Messages, Groups, Notifications, Membership, Personal Matchmaking, Settings, Overview	1–8 s	URL change **26–80 ms**, content ready **157–443 ms**
Public: How it works, Search, Membership, Success stories, Safety, About, Blog, Home	0.7–2.5 s	**26–218 ms**
Member APIs (recommendations, search, member profile, chats, notifications, shortlist, membership)	up to ~1 s	**110–200 ms**

> **Kyun pehli baar dheema?** `npm run dev` har page ko pehli baar khulne par compile karta hai (1,000–1,400 modules). Yeh sirf developer machine par hota hai. Production build (`npm run build` + `npm start`) mein pages pehle se compiled hote hain.

> **Turbopack try kiya:** `npm run dev -- --turbo` se pehli baar ka compile ~2.6x tez hua, lekin har API call par ~2 s extra lagta tha. Isliye normal `npm run dev` hi recommended hai.

3. Full-site test with a new account

Step	Result	Detail
Signup at `/signup` (Groom, Hindu)	OK	Account created and signed in. OTP step skipped because local OTP policy is OFF (see section 10).
Profile filled through the website forms	OK	25% → 75% → 100%
Photo upload (JPEG 128 kB from the browser)	OK	Stored and served as **WebP 47 kB** (900×1200), became main photo
Partner preferences	OK	Profile became "Ready for visibility", approval PENDING (manual approval is on)
Super Admin → Visibility Queue → Approve	OK	Profile then appeared in public search (men: 54 → 55)
Recommendations / Search	OK	12 recommendations, 20 search results
Open a member profile, shortlist add / list / remove	OK	
Send interest without a membership	Correctly blocked	"An active membership is required to send new interests."
Chats, notifications, membership, Personal Matchmaking pages	OK	No error screen, no console errors (except the expected 403 above)

4. Images and WebP

- **New uploads:** every member photo is converted on the server to WebP (max side 2048 px, quality 82, phone rotation fixed). Only the WebP is stored.
- **Old photos:** 10 old JPEG/PNG photos converted with `scripts/convert-photos-to-webp.ts` (saved ~4.2 MB; originals kept). 4 broken placeholder PNGs from the old dev-tools were replaced with avatars. Now **116 / 116**

photos are WebP.

- **Website images** (hero, blog, stories): served through `next/image`, which sends WebP to browsers that support it (65 kB JPEG → 21 kB WebP) and JPEG to old browsers.
- **Link preview image** `public/og-image.jpg` (1200×630) was missing — now created. Kept as JPEG because some chat apps do not show WebP previews.
- **No photo?** A default illustrated avatar (by gender + religion) is shown. It is *not* a photo: a real photo is still required to appear in search.

5. Fixed during this testing

Problem found	Fix
`sitemap.xml` and `robots.txt` missing (404)	Added `src/app/sitemap.ts` and `src/app/robots.ts` (public pages + blog; admin, dashboard, API, checkout hidden from search engines)
No public link to Personal Matchmaking	Added to the footer "Matrimonial" column
Dashboard said "No active membership" to members who have matchmaking access	Membership widget now shows "Member access — Active, included with Personal Matchmaking until …"
"Test Community" visible in the community dropdown	Came from dev-tools using a test helper. Deactivated, profiles moved to a real community, dev-tools fixed
`npm run dev:setup` would fail on photo upload (fake PNG)	dev-tools now uploads a real avatar image
Old photos not WebP; OG image missing	See section 4
Two or three `next dev` servers running in the same folder → "ChunkLoadError"	Only one dev server per project folder (see troubleshooting)

6. Still open / to decide

- **98 changed files are not committed to git yet** — must be done before handover (section 7).
- Real SMS / e-mail OTP, Razorpay/PhonePe live payments, S3 storage and backups need real accounts and a staging test.
- Load test on a real server (hundreds of users at once) not done yet.
- Older docs `LOCAL_SETUP.md` / `GIT_HANDOVER.md` show some old numbers (19 migrations). This PDF has the current ones (22 migrations, 71 tables).
- Header navigation has no "Personal Matchmaking" link (footer has one) — design decision.

Part B — Developer Handover

Clone from git, run locally, understand OTP, deploy to a server — step by step

7. FIRST: owner must commit and push the latest work

> **Zaroori:** Abhi project folder mein **98 files commit nahi hui hain** (Personal Matchmaking, Introductions, Invitations, Company Verified, WebP conversion, avatars, migrations `0019–0021`, sitemap, tests…). Naya developer agar git se lega to yeh sab **usko nahi milega**. Isliye pehle owner yeh kare:

```
# project folder mein (owner ki machine par)
npm run hooks:install              # secret-scanner pre-commit hook (once)
git status --short                 # .env, .storage-local, node_modules dikhne NAHI chahiye
npm run scan:secrets               # koi password/key file mein to nahi
git add -A
git commit -m "Personal Matchmaking, introductions, invitations, WebP, avatars, sitemap"
git push origin main
```

Git mein kabhi nahi jaata: `.env`, `.env.test`, `.storage-local/` (uploaded photos), `node_modules/`, `.next/`, database. Naya developer apni `.env` khud banayega aur apna database khud seed karega.

8. Software needed

Software	Version	Notes
Node.js	20 LTS or 22 (also works on 24)	`node -v`
npm	10+	Use npm (not yarn/pnpm) — `package-lock.json`
PostgreSQL	16 or newer	Note the `postgres` user's password; `psql` on PATH
Git	any	Access to the private GitHub repo
Git Bash (Windows)	optional	Only for `scripts/*.sh` (backup/deploy)

9. Run the project locally — step by step

1. **Code lo**

```
git clone https://github.com/uiuxathar123/RishteyZindagi-Website10-08-2026.git rishteyzindagi
cd rishteyzindagi
npm ci
```

2. **Do databases banao** (dev aur test alag — tests tables khaali kar dete hain)

```
psql -U postgres -c "CREATE DATABASE rishteyzindagi_dev;"
psql -U postgres -c "CREATE DATABASE rishteyzindagi_test;"
```

3. **Environment files banao**

```
copy .env.example .env              # macOS/Linux: cp .env.example .env
copy .env.test.example .env.test
```

`.env` mein kam se kam yeh bharo (baaki khaali chhod sakte ho):

Variable	Value
`DATABASE_URL`	`postgresql://postgres:<password>@localhost:5432/rishteyzindagi_dev`
`AUTH_SECRET`	long random string — `node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`
`NODE_ENV`	`development`
`PAYMENT_CREDENTIALS_ENCRYPTION_KEY`	64 hex chars — `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`
`SEED_DEV_SUPER_ADMIN`, `…_EMAIL`, `…_PASSWORD`	`true`, your e-mail, a strong password (local Super Admin)

`.env.test`: same, but `DATABASE_URL` → `rishteyzindagi_test` and `NODE_ENV=test`.

4. **Tables banao (22 migrations) aur reference data daalo**

```
npm run db:migrate
npm run db:seed
```

Test database ke liye bhi (PowerShell):

```
$env:DATABASE_URL="postgresql://postgres:<password>@localhost:5432/rishteyzindagi_test"; npm run db:migrate; npm run db:seed; Remove-Item Env:DATABASE_URL
```

5. **Local helper chalao** — Super Admin + 4 test members (complete profile, photo, membership) + OTP off:

```
npm run dev:setup -- --no-otp
npm run dev:doctor                  # read-only health report
```

6. *(Optional)* **100 realistic sample profiles** (search ko bhara hua dikhane ke liye):

```
npx tsx scripts/seed-sample-profiles.ts --target=dev
# hatane ke liye:  npx tsx scripts/seed-sample-profiles.ts --target=dev --remove
```

7. **Website chalao**

```
npm run dev                         # http://localhost:3000
```

Open	Login
Website `/` · Member area `/dashboard`	Test members (dev-only): `test.priya@example.com`, `test.rahul@example.com` … password `TestPass@12345`
Admin panel `/admin/login`	The e-mail/password you set in `SEED_DEV_SUPER_ADMIN_*`
Health	`/api/v1/health`, `/api/v1/health/ready`

8. **Checks** (code badalne ke baad)

```
npx tsc --noEmit                  # type check
npm run lint
npm test                          # full suite ~20 min, uses rishteyzindagi_test only
```

> **Ek folder = ek dev server.** Ek hi project folder mein do `npm run dev` mat chalao (port 3000 aur 3001). Dono same `.next` folder likhte hain aur pages "ChunkLoadError" dene lagte hain. Galti se ho jaaye to dono band karke ek hi chalao.

10. OTP — local mein bypass kyun karte hain, aur kaise

10.1 Kyun?

Signup ke waqt member ke e-mail/mobile par 6-digit OTP jaata hai. OTP bhejne ke liye ek **asli vendor** chahiye — e-mail ke liye SMTP account, SMS ke liye Fast2SMS. Developer ki machine par yeh accounts/keys nahi hote (aur hone bhi nahi chahiye), isliye OTP kabhi pahunchega hi nahi aur signup aage nahi badhega. Isliye **sirf local machine par** OTP ko bypass karte hain taaki signup, login aur baaki features test ho sakein.

> **Production mein OTP hamesha ON rakho.** OTP band karne se naye accounts ka e-mail/mobile "unverified" rehta hai — fake accounts ka khatra. Password reset, admin login/MFA aur contact change hamesha OTP maangte hain, chahe yeh setting kuch bhi ho.

10.2 Teen tareeke (local)

Tareeka	Kaise	Kab use karein
A. OTP policy OFF (sabse aasaan)	`npm run dev:otp -- off` (ya `npm run dev:setup -- --no-otp`). Wapas ON: `npm run dev:otp -- on`. Same switch: Admin → Platform Settings → "Registration OTP verification".	Roz ka local development
B. Development mode (OTP screen par dikhta hai)	Admin → Platform Settings → One-time passwords → Development mode ON. Signup ke code screen par "Show my verification code" dabao. Production mein yeh route 404 deta hai.	OTP wala screen/flow test karna ho
C. Asli e-mail OTP	`.env` mein `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `EMAIL_FROM` (local ke liye Mailpit/MailHog bhi chalega). SMS: `FAST2SMS_*` (see `FAST2SMS_SETUP.md`). Policy ON rakho.	Staging/production, ya delivery test

Setting database mein save hoti hai (`platform_settings` → key `otp.policy`), restart ke baad bhi rehti hai, aur har badlav audit log mein jaata hai.

11. Payments locally

- **Local:** Razorpay/PhonePe keys na hon aur `NODE_ENV=development` ho to checkout built-in **dev gateway** use karta hai (screen par likha aata hai). Membership aur Personal Matchmaking dono isse test ho jaate hain.
- **Asli flow test:** Razorpay ke **test-mode** keys Admin → Platform Settings → Payment gateway mein daalo (`PAYMENT_SETUP.md`, `PROVIDER_TEST_SETUP.md`).

- **Production:** live keys sirf server ke environment / admin screen mein, kabhi git mein nahi. Test keys production mein refuse hoti hain.

12. Deploy to a server (production) — step by step

1. **Server:** Linux VPS/cloud with Node 20+, PostgreSQL 16+ (managed better), nginx/Caddy for HTTPS, a private S3-compatible bucket (AWS S3, Cloudflare R2, DO Spaces…), cron.
2. **Code:** `git clone …` → `npm ci`
3. **Environment** (host ke secret store mein, `.env` file git mein nahi): `NODE_ENV=production`, `DATABASE_URL`, `APP_URL=https://your-domain`, `AUTH_SECRET`, `PAYMENT_CREDENTIALS_ENCRYPTION_KEY`, `ALLOWED_ORIGINS`, `TRUST_PROXY_HOPS=1`, `STORAGE_PROVIDER=s3` + `S3_*`, `SMTP_*` and/or `FAST2SMS_*`, `CRON_SECRET`. Full list: `PRODUCTION_ENV_REFERENCE.md`.

   ```
   npm run check:prod-env             # must print RESULT: OK
   ```

4. **Build, database, first admin, start**

   ```
   npm run build
   npm run db:migrate                 # 0000 … 0021
   npm run db:seed                    # roles, reference data, PLACEHOLDER plan prices
   npm run admin:create-super -- --email owner@your-domain --name "Site Owner"
   npm start                          # keep running with pm2 / systemd
   ```

5. **Before opening the site:** Admin → Membership Plans: set real prices · Admin → Personal Matchmaking → Settings: price + on-sale · Payment gateway live keys · OTP policy ON.
6. **nginx/HTTPS:** forward to `127.0.0.1:3000`, pass `Host`, `X-Forwarded-For`, `X-Forwarded-Proto`; block `/api/v1/internal/` publicly.
7. **Webhooks:** Razorpay `https://<domain>/api/v1/webhooks/razorpay`, PhonePe `…/webhooks/phonepe`.
8. **Cron jobs** (`npm run job -- --list`):

   ```
   */15 * * * *  cd /srv/rishteyzindagi && npm run job -- membership-expiry
   5    * * * *  cd /srv/rishteyzindagi && npm run job -- membership-reminders
   */30 * * * *  cd /srv/rishteyzindagi && npm run job -- membership-renewal-failed
   */10 * * * *  cd /srv/rishteyzindagi && npm run job -- payment-reconcile
   30   2 * * *  cd /srv/rishteyzindagi && npm run job -- account-deletion-finalize
   17   * * * *  cd /srv/rishteyzindagi && npm run job -- housekeeping
   30   3 * * 1  cd /srv/rishteyzindagi && npm run job -- member-digest
   15   * * * *  cd /srv/rishteyzindagi && npm run job -- membership-dunning
   ```

9. **After deploy:** `curl -f https://<domain>/api/v1/health/ready` → 200 · signup with real OTP · upload a photo · one test payment · run a backup + restore check (`BACKUP_DR.md`).

12.1 Later updates (every new version)

```
git pull
npm ci
scripts/backup-secure.sh           # backup BEFORE migrating
npm run db:migrate
npm run build
# restart the app (pm2 restart / systemctl restart)
```

13. Tests

- `npm test` runs only against the database in `.env.test` and refuses anything that is not a local `rishteyzindagi_test`. It empties tables — never point it at real data.
- A single file: `npx vitest run tests/matchmaking-payments.test.ts`
- After changing `src/server/db/schema.ts`: `npm run db:generate` → new file in `drizzle/` → `npm run db:migrate` on dev and test DB.

14. Troubleshooting

Problem	Solution
`DATABASE_URL is not set` / Invalid environment	`.env` missing or not in the project root
"relation … does not exist"	`npm run db:migrate` on the right database
ChunkLoadError / pages stuck loading	More than one `next dev` in the same folder — stop all, start one
Port 3000 busy	Stop the old server, or `npx next dev -p 3001` (only one server!)
Signup says service unavailable / no OTP arrives	Section 10 — policy OFF or dev mode ON locally
Search is empty	Profile must be complete + photo + approved + visible; `npm run dev:doctor` explains; or seed sample profiles
Interest says membership required	Correct — member needs an active plan (dev-setup members have one)
Admin login asks to change password	Temporary password from `admin:create-super` — set your own
Tests fail at start	PostgreSQL not running, wrong `.env.test`, or test DB not migrated/seeded

15. Useful commands

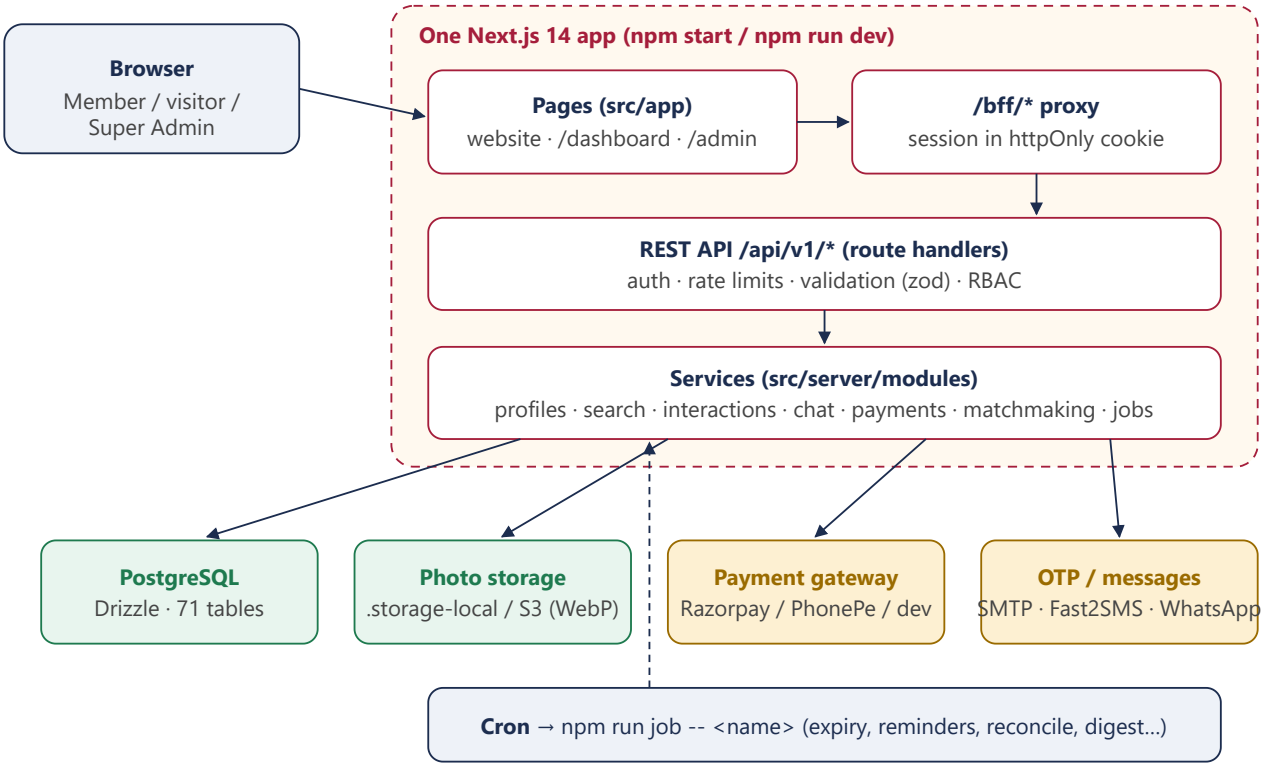
Command	What it does
`npm run dev` / `build` / `start`	Run locally / production build / production server
`npm run db:migrate` / `db:seed` / `db:generate`	Apply migrations / reference data / new migration from schema
`npm run dev:setup -- --no-otp` · `dev:doctor` · `dev:otp -- on|off`	Local helpers (refuse to run in production / remote DB)
`npx tsx scripts/seed-sample-profiles.ts --target=dev [--remove]`	100 realistic sample profiles (local only)
`npx tsx scripts/convert-photos-to-webp.ts --target=dev [--dry-run]`	Convert old JPEG/PNG photos to WebP
`npx tsx scripts/make-og-image.ts`	Recreate the link-preview image
`npm run admin:create-super -- --email … --name "…"`	First Super Admin (production)
`npm run job -- --list` · `npm run job -- <name>`	Scheduled jobs
`npm run check:prod-env` · `scan:secrets`	Production env check · secret scanner

More detail: `LOCAL_SETUP.md`, `DEPLOYMENT_PRODUCTION.md`, `PAYMENT_SETUP.md`, `BACKUP_DR.md`, `API_REFERENCE.md`, `BACKEND.md`.

Part C — Project Flow Charts

How the system is built and how users, money and data move through it

16. System architecture



Browser kabhi seedha API token nahi rakhta: pages /bff proxy se baat karte hain, jo cookie se session lagata hai. Sab kuch ek hi Next.js app mein hai.

17. Member journey

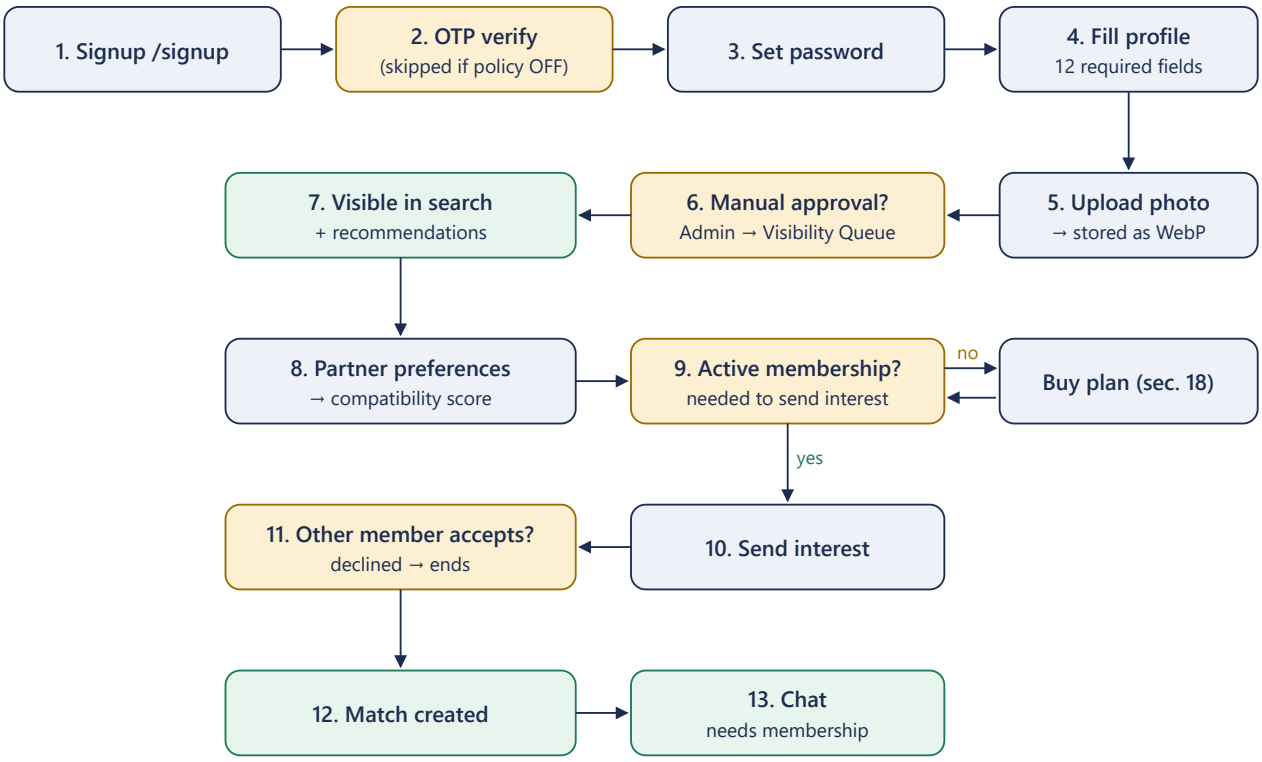
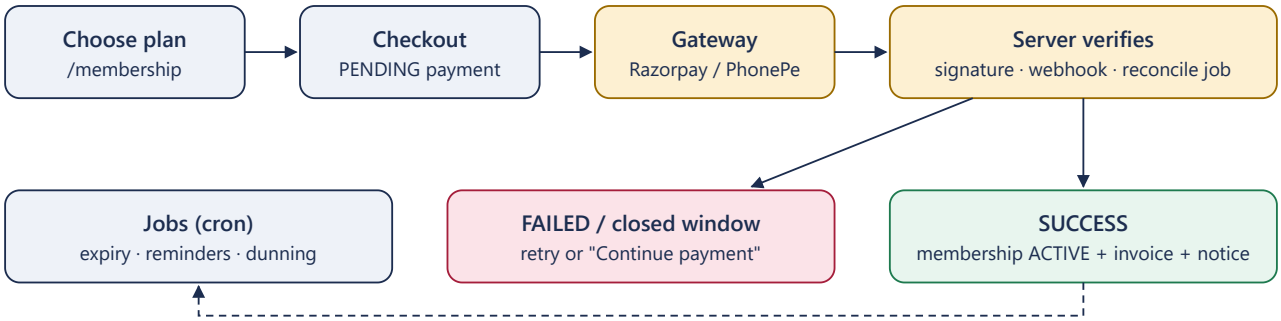


Photo nahi hai to profile par default avatar dikhta hai, lekin search mein aane ke liye asli photo zaroori hai. Interest bhejne aur chat ke liye active membership (ya Personal Matchmaking access) chahiye.

18. Membership payment flow



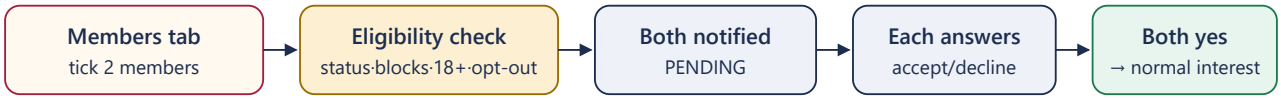
Browser kabhi payment ko "paid" nahi bata sakta: sirf server ka verify (signature / webhook / reconcile) payment ko SUCCESS karta hai. Local mein bina keys ke "dev gateway" yahi flow simulate karta hai.

19. Personal Matchmaking (Super Admin)

A. Paid service



B. Introductions (no payment needed)

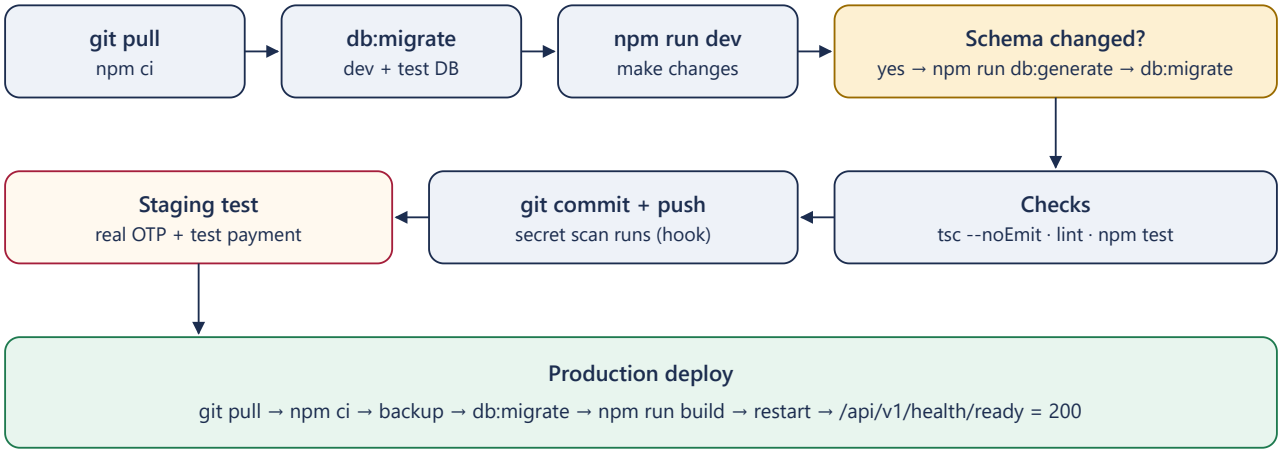


C. Invitations and Company Verified



Introduction kabhi apne aap match ya chat nahi kholti. Sab actions sirf Super Admin kar sakta hai aur audit log mein jaate hain.

20. Developer workflow (code change → live)



Har migration se pehle production database ka backup. Migrations sirf aage badhti hain (additive) — rollback DEPLOYMENT.md §7 mein.

> **Shuruaat ka sabse chhota raasta (naya developer):** section 7 ka commit/push confirm karo → section 9 ke steps 1–7 → `/admin/login` aur `test.priya@example.com` se login karke dekho → section 12 deploy ke liye.